



CIN0136

Desenvolvimento de Software

Desenvolvimento de Software

Prof. Vinicius Cardoso Garcia

vcg@cin.ufpe.br :: @vinicius3w :: assertlab.com

<http://viniciusgarcia.me>



**Centro de
Informática**
UFPE



UNIVERSIDADE
FEDERAL
DE PERNAMBUCO

Licença do material

Este Trabalho foi licenciado com uma Licença



**Atribuição-NãoComercial-
Compartilha Igual 4.0 Internacional
(CC BY-NC-SA 4.0)**

Mais informações visite

<https://creativecommons.org/licenses/by-nc-sa/4.0/deed.pt> BR



Engenharia de Software em Dimensões

É um material abrangente, didático e atualizado que apresenta a Engenharia de Software como uma disciplina multidimensional. Cada capítulo foca em uma dimensão prática, conectando teoria e realidade profissional com clareza, profundidade e propósito.

Prof. Vinicius Cardoso Garcia

<https://bit.ly/esdbook>

Engenharia de Software em Dimensões

Da base conceitual à prática profissional



Vinicius Cardoso Garcia



Ponto de Commit

O podcast figital da disciplina (IF977) Engenharia de Software do curso de bacharelado em Sistemas de Informação do Centro de Informática (CIn) da Universidade Federal de Pernambuco (UFPE).

<https://go.uaite.xyz/ponto-de-commit>

Ponto de Commit



O podcast figital da disciplina de Engenharia de Software



Engenharia de Software Moderna

Engenharia de Software Moderna é um livro destinado a alunos de cursos de graduação em Computação. Ele pode ser lido também por profissionais que buscam um conhecimento horizontal da área.

Prof. Marco Tulio Valente

<https://engsoftmoderna.info>

@engsoftmoderna

ENGENHARIA — DE — SOFTWARE MODERNA

Princípios e práticas para
desenvolvimento de software
com produtividade



MARCO TULIO VALENTE



Requisitos Funcionais, Não Funcionais e Técnicas de Elicitação

Da conversa informal ao artefato formal,
estruturando o que sua equipe já faz
intuitivamente



Objetivos de Aprendizagem

01

Classificar requisitos

Distinguir requisitos funcionais de não funcionais e classificar corretamente exemplos concretos do seu próprio projeto.

03

Reconhecer requisitos informais

Identificar, no próprio projeto de extensão, requisitos já levantados que ainda não foram formalizados.

02

Nomear técnicas de elicitação

Identificar pelo menos três técnicas e indicar quando cada uma é mais adequada ao contexto.

04

Formular boas perguntas

Elaborar perguntas de elicitação que vão além do óbvio, e revelam o que o cliente não sabe que precisa.



Exercício de Abertura: "Descreva o que o WhatsApp faz"

A instrução

Você tem 2 minutos. Escreva tudo que o WhatsApp faz. Seja específico.



Exercício de Abertura: "Descreva o que o WhatsApp faz"

A instrução

Você tem 2 minutos. Escreva tudo que o WhatsApp faz. Seja específico.

As perguntas que seguem

As listas são iguais? O que está numa e não está nas outras? Por que isso acontece?



Exercício de Abertura: "Descreva o que o WhatsApp faz"

A instrução

Você tem 2 minutos. Escreva que o WhatsApp faz. Seja específico.

As perguntas que seguem

As listas são iguais? O que está numa e não está nas outras? Por que isso acontece?

O que o exercício revela

- Cada pessoa foca em aspectos diferentes: uso pessoal, técnico, negócio

Alguns descrevem *o que faz* (enviar mensagem), outros *como faz* (criptografia), outros *como parece* (interface limpa)

- Ninguém cobre tudo — e quase ninguém descreve requisitos não funcionais explicitamente



Fechamento do bloco: Se cada pessoa da equipe tem uma lista diferente na cabeça, o que acontece durante o desenvolvimento? É exatamente isso que requisitos mal definidos causam, e é por isso que Engenharia de Requisitos existe.



Por Que Precisamos de Requisitos Formais?



Comunicação ambígua

Sem linguagem comum, cada membro da equipe implementa sua própria versão do sistema imaginado.



Retrabalho caro

Corrigir um requisito mal definido na semana 10 pode custar 10× mais do que corrigi-lo na semana 2.



Base para validação

Sem requisitos claros, não há critério objetivo para dizer se o sistema está "pronto" ou "correto".



Requisitos Funcionais vs. Não Funcionais

Requisito Funcional

Descreve o que o sistema deve fazer:

comportamentos, funções e serviços oferecidos. É diretamente verificável: ou o sistema faz, ou não faz.

Exemplo: "O usuário pode enviar uma mensagem de texto."

Requisito Não Funcional

Descreve **como** o sistema deve se comportar — qualidade, desempenho, segurança, usabilidade.

Também chamado de **atributo de qualidade**.

Exemplo: "O sistema deve suportar 2 bilhões de usuários simultâneos."

A distinção parece simples — mas na prática, a categorização depende de **como o requisito é escrito**, não apenas do tema. Um mesmo assunto (ex: criptografia) pode ser funcional ou não funcional dependendo da formulação.



Quiz Rápido: Funcional ou Não Funcional?

Classifique cada item em voz alta. Preste atenção nos casos ambíguos — eles ensinam mais do que os óbvios.

Requisito	Classificação
O usuário pode enviar uma mensagem de texto.	
O sistema deve suportar 2 bilhões de usuários simultâneos.	
O usuário pode criar grupos de até 1.024 participantes.	
As mensagens devem ser criptografadas de ponta a ponta.	
A interface deve ser acessível para pessoas com deficiência visual.	



Quiz Rápido: Funcional ou Não Funcional?

Classifique cada item em voz alta. Preste atenção nos casos ambíguos — eles ensinam mais do que os óbvios.

Requisito	Classificação
O usuário pode enviar uma mensagem de texto.	Funcional
O sistema deve suportar 2 bilhões de usuários simultâneos.	Não funcional (desempenho)
O usuário pode criar grupos de até 1.024 participantes.	Funcional
As mensagens devem ser criptografadas de ponta a ponta.	⚠️ Ambíguo — depende da formulação
A interface deve ser acessível para pessoas com deficiência visual.	Não funcional (usabilidade)

- 📝 🔥 O item 4 é propositalmente ambíguo. Use o debate que ele gera para mostrar que a categorização depende de como o requisito é escrito. Se descrito como comportamento verificável, pode ser funcional. Se descrito como atributo de qualidade, é não funcional.



Categorias de Requisitos Não Funcionais



Desempenho

Tempo de resposta, throughput, latência. Ex: "A mensagem deve ser entregue em menos de 200ms em condições normais de rede."



Segurança

Autenticação, autorização, criptografia, privacidade de dados.
Frequentemente negligenciados até que algo dê errado.



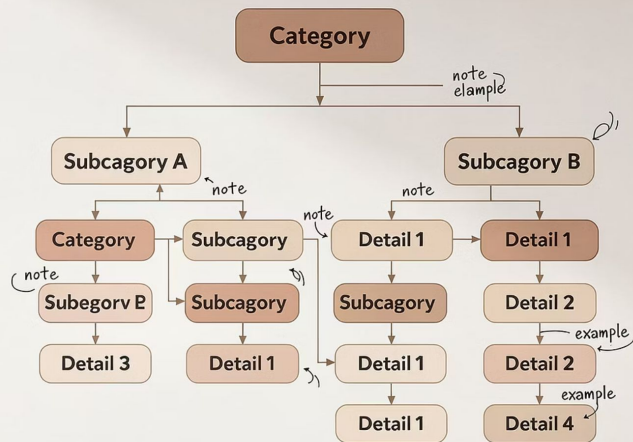
Usabilidade

Facilidade de aprendizado, eficiência de uso, acessibilidade. Difíceis de medir, mas decisivos para adoção.



Escalabilidade

Capacidade de crescer com a demanda sem degradação. Muito mais barato planejar agora do que refatorar depois.



Técnicas de Elicitação

As técnicas abaixo têm nome formal, mas provavelmente vocês já usaram algumas delas sem perceber. O objetivo agora é ampliar o repertório e tornar o uso **intencional e eficaz**.

1

Entrevistas

A mais usada e a mais mal feita

2

Workshops

Para múltiplos stakeholders com perspectivas conflitantes

3

Observação

Ver o usuário em ação no ambiente real

4

Prototipagem

O protótipo como pergunta visual ao cliente



Técnica 1: Entrevistas

A técnica mais usada — e a mais frequentemente mal executada. O erro clássico é perguntar **"o que você quer?"** em vez de explorar o problema. Perguntas que abrem território:

"Me conta como você faz isso hoje."

Revela o processo atual, não apenas o desejo. Você aprende o contexto real, não o imaginado.

"O que acontece quando dá errado?"

Revela casos de falha não óbvios — que raramente aparecem em briefings ou documentos formais.

"Quem mais usa isso?"

Revela stakeholders esquecidos que também têm necessidades legítimas — e que podem bloquear a adoção.

"O que tornaria isso um sucesso para você?"

Revela o critério de aceitação implícito — o que o cliente usa para decidir se ficou bom ou ruim.



O Poder de Uma Boa Pergunta

✗ Pergunta pobre

"O que você quer que o sistema faça?"

Convida o cliente a especular sobre soluções que ele não tem expertise para descrever.

✓ Pergunta rica

"Me mostra como você resolve esse problema hoje."

Ancora a conversa na realidade vivida — e revela necessidades que o cliente nem sabia que tinha.



Técnica 1: Entrevistas

Entrevistas Estruturadas

Seguem um roteiro pré-definido de perguntas, ideal para coletar informações específicas e comparáveis entre diferentes stakeholders.

Entrevistas Não Estruturadas

São mais abertas e exploratórias, permitindo que o entrevistado guie a conversa e revele informações inesperadas.

Entrevistas Semiestruturadas

Combinam perguntas pré-definidas com a flexibilidade de explorar tópicos que surgem durante a conversa.

Entrevistas em Grupo

Reúnem vários stakeholders simultaneamente, promovendo discussões e revelando dinâmicas e conflitos entre diferentes perspectivas.



Técnica 2: Workshops de Requisitos



Quando usar

Quando há **múltiplos stakeholders com perspectivas conflitantes**. O problema é político além de técnico.

O que faz diferença

- Cria um espaço neutro para negociação explícita entre partes com interesses divergentes
- Permite que conflitos de requisitos venham à tona cedo — e não na entrega
- Facilita a construção de consenso com documentação imediata

☐ Para os projetos deste semestre, provavelmente não será necessário — mas é importante conhecer para projetos futuros.



Técnica 2: Workshops de Requisitos



Brainstorming

Sessões criativas para gerar um grande número de ideias e soluções potenciais.



JAD (Joint Application Development)

Reuniões estruturadas que juntam usuários e desenvolvedores para definir requisitos em conjunto.



Priorização

Atividades para classificar e ordenar requisitos com base em sua importância e viabilidade.



Prototipagem Rápida

Criação de modelos simples para visualizar e validar ideias rapidamente.



Técnica 3: Observação

Observar o usuário trabalhando no seu **ambiente natural** — sem perguntar, apenas assistindo. Revela o que ninguém pensa em mencionar porque considera óbvio.

O exemplo clássico

Observar uma secretária usando um sistema legado e perceber que ela tem um **caderninho ao lado do computador** com anotações que o sistema não suporta. O caderninho é o requisito não documentado.

Pergunta para a turma

Alguém visitou o espaço de trabalho do parceiro? Observou como eles trabalham hoje? O que viu que **não estava no briefing**?



Técnica 3: Observação

1

Observação Passiva

O analista observa os usuários em seu ambiente de trabalho sem interferir, registrando suas atividades e fluxos de trabalho.

2

Observação Ativa

O analista interage com os usuários durante a observação, fazendo perguntas e solicitando explicações sobre processos específicos.

3

Etnografia

Uma abordagem mais imersiva, onde o analista se integra ao ambiente do usuário por um período prolongado para entender profundamente o contexto e a cultura.

4

Análise de Protocolos

Os usuários são solicitados a "pensar em voz alta" enquanto realizam tarefas, permitindo ao analista entender seus processos de pensamento e tomada de decisão.



Técnica 4: Prototipagem como Elicitação

A lógica central: muitos clientes **não conseguem descrever o que querem em palavras**, mas reconhecem imediatamente quando veem — ou rejeitam — algo concreto. O protótipo não é apenas para mostrar uma solução. É uma **pergunta visual**.

1

Você cria o protótipo

Um esboço, wireframe ou mockup baseado no que entendeu até agora

2

O cliente reage

"Não é bem isso..." — essa rejeição é informação valiosa, não fracasso

3

Você descobre o requisito

O cliente revelou uma necessidade que não sabia que tinha — e que nunca descreveria em palavras

- 📌 🎯 Quando vocês forem apresentar o protótipo hi-fi ao parceiro, preparem-se para ouvir "não é bem isso". Entendam isso como **informação**, não como fracasso. Esse momento é elicitación.



Técnica 4: Prototipagem como Elicitação

Protótipos de Baixa Fidelidade

Esboços rápidos ou wireframes que representam a estrutura básica e o fluxo da interface do usuário. Úteis para validar conceitos iniciais e layouts gerais.

Protótipos de Alta Fidelidade

Representações mais detalhadas e interativas do sistema, muitas vezes criadas com ferramentas especializadas. Permitem uma experiência mais próxima do produto final.

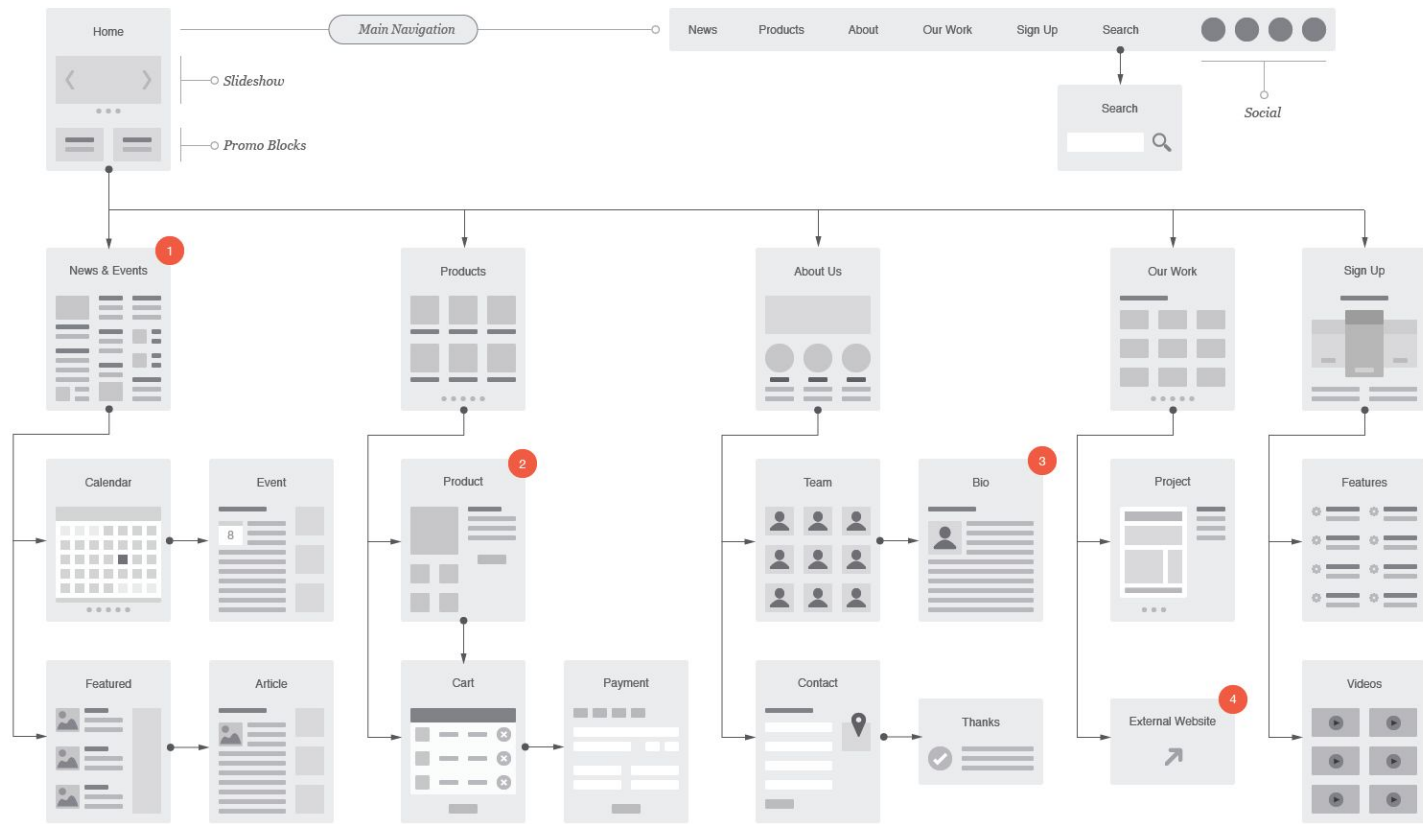
Protótipos Descartáveis

Criados rapidamente para explorar ideias específicas e depois descartados. Focam na velocidade de criação e teste de conceitos.

Protótipos Evolutivos

Começam simples e são iterativamente refinados, eventualmente evoluindo para o sistema final. Úteis em metodologias ágeis e desenvolvimento incremental.





1 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur massa urna, sollicitudin sit amet ante at, aliquet tincidunt enim. Cras dapibus non nisi.

2 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur massa urna.

3 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur massa urna, sollicitudin sit amet ante at, aliquet tincidunt enim. Cras dapibus non nisi.

4 Lorem ipsum dolor sit amet, consectetur adipiscing elit. Curabitur massa urna.



Comparativo das Técnicas

Cada técnica tem seu momento ideal. Conhecer o repertório é o que permite escolher a ferramenta certa para cada situação.

Técnica	Melhor quando...	Cuidado com...
Entrevista	Poucos stakeholders, problema pouco estruturado	Perguntas fechadas que confirmam suposições
Workshop	Múltiplos stakeholders, conflitos de perspectiva	Dominância de uma voz que silencia as demais
Observação	Processos complexos, usuários não verbalizáveis	Presença do observador alterar o comportamento
Prototipagem	Clientes que "não sabem" o que querem até ver	Cliente validar o protótipo em vez de avaliar



A Pergunta Difícil da Elicitação

Existe informação que o stakeholder tem mas não vai te contar voluntariamente?

Por que isso acontece?

- O stakeholder considera "óbvio demais para mencionar"
- Há interesse político em omitir certas informações
- A linguagem técnica cria barreiras de comunicação
- O cliente pede o que quer, não o que precisa

Como chegar lá?

- Observação direta no ambiente de trabalho
- Perguntas indiretas sobre processo e falhas
- Entrevistas com múltiplos stakeholders e triangulação
- Protótipo que força reação concreta, não descrição abstrata



Questão Estruturante: A Tensão Central

"Qual é o risco de começar a implementar sem requisitos bem definidos? E qual é o risco de esperar demais para começar a implementar enquanto tenta definir todos os requisitos?"



Questão Estruturante: A Tensão Central

"Qual é o risco de começar a implementar sem requisitos bem definidos? E qual é o risco de esperar demais para começar a implementar enquanto tenta definir todos os requisitos?"

⚠ Implementar sem requisitos

- Retrabalho caro nas etapas finais
- Equipe alinhada a versões diferentes do sistema
- "Funciona, mas não é o que eu queria"
- Débito técnico difícil de rastrear à origem

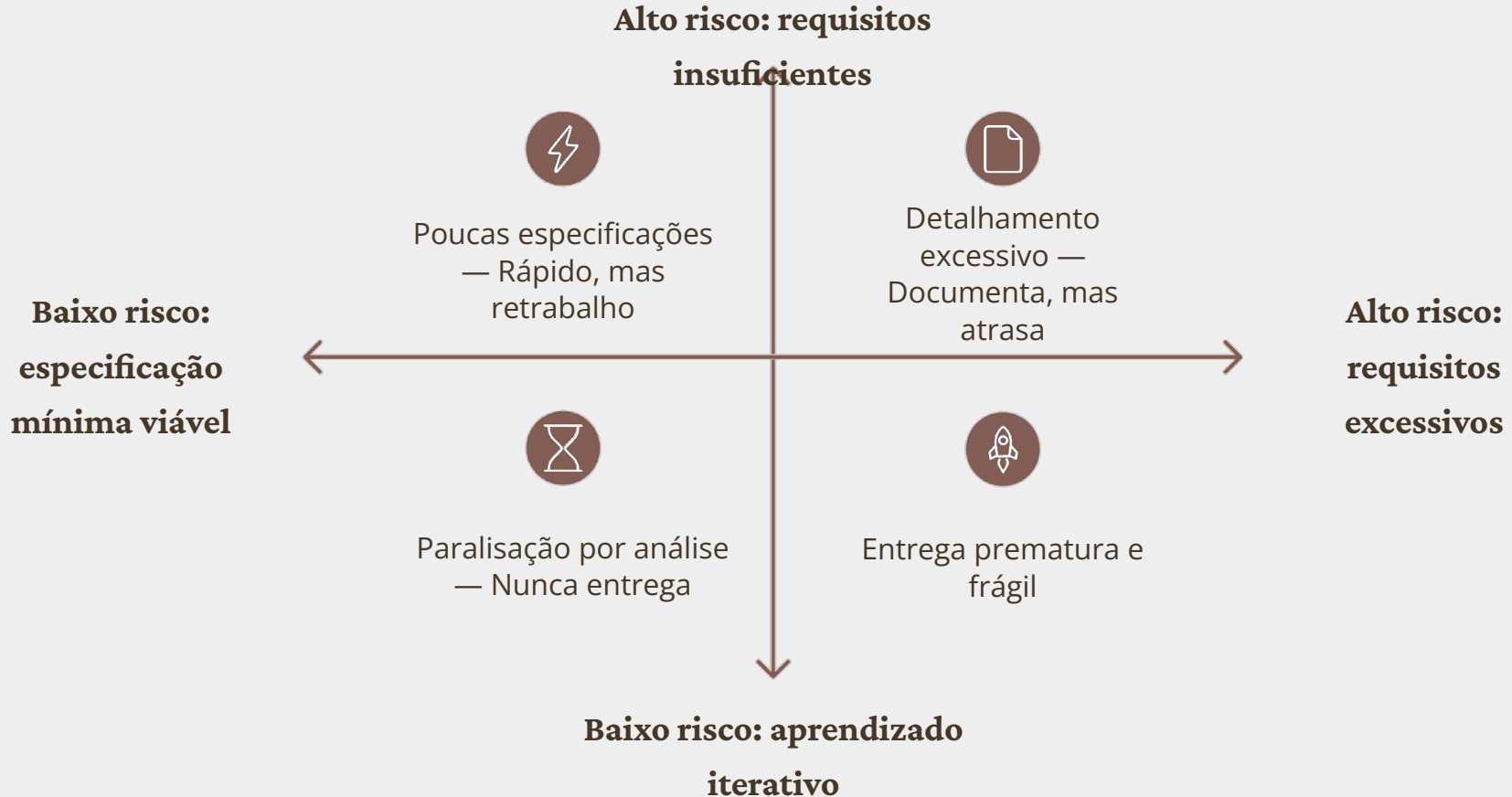
⚠ Esperar demais para implementar

- Análise paralisante — requisitos nunca estão "completos"
- Feedback real chega tarde demais para ajustes
- Cliente muda de ideia — o documento ficou obsoleto
- Tempo desperdiçado especificando o que não será construído

❏ Não há resposta certa, o objetivo é perceber que existe um **equilíbrio difícil a calibrar**. Em contextos ágeis, a resposta é iteração: especifique o suficiente para começar, implemente, aprenda, refine.



A Tensão em Contextos Ágeis





Em metodologias ágeis, a resposta para essa tensão **não é escolher um lado, é iterar**. Especifique o suficiente para que a equipe comece com confiança. Implemente. Aprenda com o feedback real. Refine os requisitos. Repita.



Fechamento e Próxima Aula



Amanhã vocês vão aprender a transformar o que entenderam hoje em **artefatos formais**: user stories, critérios de aceitação e um backlog priorizado.



Tarefa para amanhã

Tragam as stories do projeto escritas no formato:

"Como [persona], quero [funcionalidade] para [benefício]."

Pelo menos três por pessoa.



Leitura prévia para terça

Cap. 7, seções 7.1.2 a 7.2.4

Valente, Cap. 3

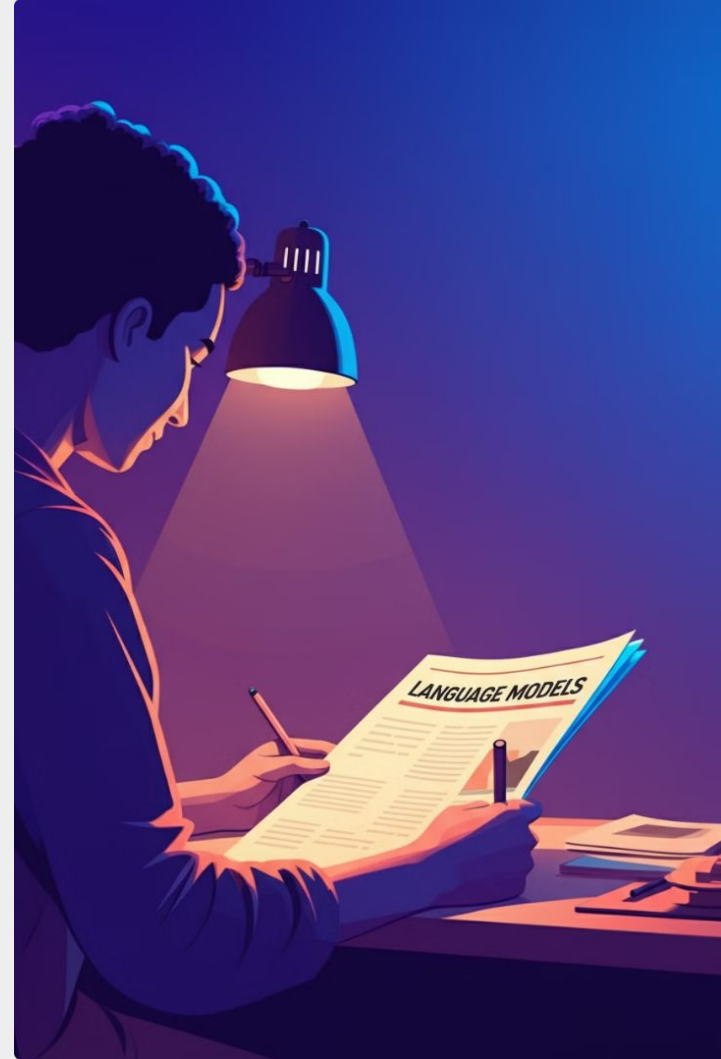
Foco: critérios de aceitação e backlog priorizado.

"Requisitos são a linguagem comum entre quem entende o problema e quem constrói a solução. Sem essa linguagem, cada um constrói um sistema diferente na própria cabeça."



Próximos passos

- Eng. Software em Dimensões
 - Cap. 6 (seções 6.1–6.2):
Requisitos funcionais e não funcionais, elicitação
- Eng. Software Moderna
 - Cap. 3 —
Requisitos Histórias de Usuário, MVP, priorização



contacts

vcg@cin.ufpe.br
[@vinicius3w](https://www.instagram.com/vinicius3w)
viniciusgarcia.me

academy

go.uaite.xyz/vcg-es
go.uaite.xyz/vcg-microservices
go.uaite.xyz/vcg-devops
go.uaite.xyz/vcg-esaia
go.uaite.xyz/vcg-deso

Desenvolvimento de Software (CIN0136)

Requisitos Funcionais, Não
Funcionais e Técnicas de
Elicitação